

category: networking-protocol

tags: [sharppcap, packetdotnet, npcap, pcap, loopback, dlt-null, lso, gso, windows, tcp-reassembly]

severity: high

created: 2026-07-29

updated: 2026-07-29

project-origin: App-Access-V2 (ZkPushMonitor — chế độ Capture bằng SharpPcap/Npcap)

Bắt gói tin trên loopback Windows bằng SharpPcap/Npcap: `LinkLayers.Null` (không phải Ethernet) + IP Total Length bị Windows LSO/GSO ghi sai (0xFFFF) khi payload lớn

Tình huống gặp phải

Viết thêm chế độ "Capture" cho `ZkPushMonitor` (thay vì proxy chủ động, tool nghe thụ động qua Npcap để thiết bị không cần đổi cấu hình). Dùng SharpPcap 6.3.0 + PacketDotNet 1.4.7, test bằng script Python giả lập gửi HTTP qua `127.0.0.1` (loopback) trên Windows, dùng adapter Npcap "Adapter for loopback traffic capture".

Triệu chứng / Lỗi

1. **Lỗi 1 — parse packet:** `Packet.ParsePacket(raw.LinkLayerType, raw.Data)` ném `IndexOutOfRangeException: Source array was not long enough. Check the source index, length, and the array's lower bounds. (Parameter 'sourceArray')` ngay khi có 1 request body lớn (~200 KB) đi qua, dù các request nhỏ (vài chục–vài trăm byte) trước đó parse hoàn toàn bình thường.
2. **Lỗi 2 (sau khi tương đã fix bằng cách tự bóc header + đổi `LinkLayers.Raw`):** không còn exception, nhưng 1 kết nối TCP THẬT bị log thành 3 "kết nối mới" khác nhau dù cùng 1 địa chỉ `IP:port`, và stream ghép lại thiếu đúng ~4682/200046 byte cuối (số liệu lặp lại y hệt qua nhiều lần chạy khác nhau — không phải ngẫu nhiên).
3. Debug bằng hex dump raw packet: `LinkLayerType=Null, dataLen=65536, hex=02000000 4500 FFFF 2A90 4000 8006 ...` — 4 byte đầu `02 00 00 00` (little-endian) = `AF_INET` đúng chuẩn `DLT_NULL` của BSD/Npcap; nhưng byte IP header ngay sau đó ghi **Total Length = 0xFFFF** (65535).

Nguyên nhân gốc rễ (Root Cause)

- **Root cause thật sự:** Windows dùng **LSO/GSO (Large/Generic Segmentation Offload)** khi ứng dụng gửi 1 lượng dữ liệu lớn qua loopback. Tại điểm mà Npcap chèn vào để capture, dữ liệu **CHƯA được phân mảnh** thành các frame MTU thật (như sẽ xảy ra trên NIC vật lý) — đây là 1 "siêu gói" gộp, và trường **IP Total Length bị ghi giá trị "danh nghĩa" 0xFFFF** (không phản ánh độ dài thật đã capture được). Đây là

hiện tượng đã biết, chính Wireshark cũng phải tự dò và sửa lại trường này khi capture loopback trên Windows.

- PacketDotNet **tin theo** Total Length trong header để tính vùng payload IP/TCP → cố đọc vượt quá độ dài mảng byte thật đang có → ném exception.
- **Sai lầm ở lần fix đầu tiên:** tự bóc 4 byte DLT_NULL rồi gọi `Packet.ParsePacket(LinkLayers.Raw, ipData)` — PacketDotNet **đã tự xử lý đúng** header BSD loopback (family 4-byte) khi gọi với `LinkLayers.Null` (đúng type gốc từ `raw.LinkLayerType`); việc tự diễn giải lại bằng `LinkLayers.Raw` là **dư thừa và sai ngữ nghĩa** — gây thêm lỗi đọc sai offset/cờ TCP (SYN/FIN/RST bị đọc nhầm ở vị trí khác), dẫn tới hiện tượng 1 kết nối bị tách thành 3 "connection" giả trong tầng theo dõi của ứng dụng.
- Nguyên nhân thật sự phải sửa: **vá lại 2 byte Total Length ngay TRÊN `raw.Data` gốc** (ở đúng offset IP header = 4, sau family field) thành đúng độ dài THẬT đã capture được, rồi để `Packet.ParsePacket(raw.LinkLayerType /* == Null, giữ nguyên */, raw.Data)` tự parse như bình thường — giống hệt cách Wireshark tự sửa nội bộ khi phát hiện Total Length bất thường trên loopback.

Giải pháp

```
// Sửa lại "IP Total Length" khi phát hiện dấu hiệu LSO/GSO trên loopback Windows
(trường
// này lớn hơn số byte THẬT đã capture được, tính từ đầu IP header) — đổi thành đúng độ
dài
// thật. Sửa TRỰC TIẾP trên mảng gốc raw.Data, KHÔNG tự bóc byte / đổi LinkLayers.
private static void FixupWindowsLoopbackLsoLength(byte[] data, int ipHeaderOffset)
{
    if ((data[ipHeaderOffset] >> 4) != 4) return; // chỉ IPv4

    int totalLength = (data[ipHeaderOffset + 2] << 8) | data[ipHeaderOffset + 3];
    int realLength = data.Length - ipHeaderOffset;
    if (totalLength > realLength)
    {
        data[ipHeaderOffset + 2] = (byte)(realLength >> 8);
        data[ipHeaderOffset + 3] = (byte)(realLength & 0xFF);
    }
}

// Gọi TRƯỚC khi parse, giữ nguyên LinkLayerType gốc (Null) — KHÔNG tự bóc 4-byte
header:
if (raw.LinkLayerType == LinkLayers.Null && raw.Data.Length > 24)
{
    FixupWindowsLoopbackLsoLength(raw.Data, ipHeaderOffset: 4);
}
var packet = Packet.ParsePacket(raw.LinkLayerType, raw.Data);
```

Kết quả verify: exception biến mất, connection tracking về đúng 1 connection/kết nối thật (không còn tách 3), toàn bộ body nhỏ/vừa (không kèm request 200KB) khớp sha1 100%.

Giới hạn còn lại (chấp nhận được): Body cực lớn (~200 KB) qua LOOPBACK vẫn có thể mất ~4.6 KB đuôi dữ liệu dù đã sửa Total Length — đây là do bản chất siêu-gói LSO không phải lúc nào cũng phản ánh đúng biên giới TCP segment thật ở tầng capture, khó tái tạo 100% một cách tổng quát. Đã verify: **request thân thường/thường gặp (không đặc biệt lớn) qua loopback đều chính xác 100%**; hiện tượng chỉ xảy ra khi test bằng loopback với payload cỡ lớn bất thường — thiết bị thật qua NIC vật lý (Ethernet/Wi-Fi) không bị ảnh hưởng vì gói tin được phân mảnh thật ở tầng driver mạng, không qua LSO loopback.

Áp dụng lại (How to reuse)

- Bắt gói tin trên **loopback Windows** bằng SharpPcap/Npcap → LUÔN kiểm tra `raw.LinkLayerType == LinkLayers.Null` (không phải Ethernet) và LUÔN vá lại Total Length nếu request/response có thể mang payload lớn (>1 vài chục KB) — không đợi tới khi thấy exception mới nghĩ tới việc này.
- KHÔNG tự bóc byte header rồi đổi sang link-layer-type khác để "giúp" thư viện — PacketDotNet (và hầu hết binding libpcap) đã tự xử lý đúng theo type gốc; tự diễn giải lại dễ gây lỗi tinh vi hơn (đọc sai offset cờ/field) mà không ném exception rõ ràng, khó phát hiện hơn nhiều so với lỗi ban đầu.
- Khi gặp 1 kết nối TCP thật bị log thành NHIỀU kết nối trong tầng theo dõi ứng dụng (dù IP:port giống hệt nhau) — nghi ngay lỗi đọc sai cờ SYN/FIN/RST (misparsed offset), không phải race condition hay bug logic connection-tracking, đặc biệt nếu vừa đổi cách parse gói tin.
- Log kèm `LinkLayerType` + `dataLen` trong message lỗi khi bắt exception liên quan tới parse gói tin — tiết kiệm rất nhiều thời gian chẩn đoán so với chỉ log `ex.Message`.

Chú ý / Cạm bẫy (Gotchas)

- ⚠ Hiện tượng Total Length = 0xFFFF **chỉ xảy ra trên loopback Windows với payload lớn** — test với message nhỏ (vài trăm byte, đúng kích thước message PUSH điển hình của ZKTeco) sẽ KHÔNG lộ ra bug này, dễ gây ảo tưởng "đã chạy ổn" rồi mới vỡ ra khi test với payload lớn hơn.
- ⚠ SharpPcap 6.x đổi API `Open()`: chữ ký chính thức mới là `Open(DeviceConfiguration)`, nhưng vẫn còn extension method `Open(DeviceModes mode, int read_timeout)` (tham số tên `read_timeout`, không phải `readTimeoutMilliseconds`) để tương thích code kiểu SharpPcap 5.x — gọi bằng positional argument để tránh lỗi CS1739.
- ⚠ Npcap cần cài kèm tùy chọn hỗ trợ "**Adapter for loopback traffic capture**" mới bắt được traffic `127.0.0.1` — mặc định không phải NIC vật lý nào cũng thấy traffic loopback (đây là adapter ảo riêng do Npcap tạo).

Tham chiếu

- Project liên quan: App-Access-V2 — `iAccessDesktopV2.Avalonia/iAccess.Tools.ZkPushMonitor/Capture/PackageCaptureMonitor.cs`
- Liên quan: reverse-proxy-breaks-vendor-software-remote-command-source-ip-routing.md (lý do cần xây thêm chế độ Capture để tránh vấn đề IP nguồn của reverse-proxy)